

Requirement Gathering Specification

Date: 20 July , 2026

Project Overview

Brief Description

The Sentinel platform is an AI-assisted software engineering intelligence and project risk prediction platform. By continuously mining version control metadata, issue tracking systems, collaboration patterns, and documentation, the system maps the evolution of the software project into an engineering knowledge graph. This graph is analyzed using specialized machine learning models to predict engineering risks before they occur, offering human-interpretable explanations and prioritized refactoring recommendations.

Problem Aimed to Solve

Traditional software engineering management tools suffer from a fundamental limitation: they are descriptive and past-facing rather than predictive. Platforms such as GitHub, GitLab, Jira, and SonarQube focus on reporting historical state changes, such as the total count of commits, open issues, completed story points, or static code coverage percentages. While these metrics provide retrospection, they fail to answer critical forward-looking questions, such as which modules are likely to experience the next critical defect, which developer has become an unsustainable knowledge bottleneck, or which sprint is at risk of failure.

This reactive posture forces software organizations to address technical debt, architectural erosion, and knowledge fragmentation only after they have manifested as costly operational failures. Furthermore, static code analysis tools (such as SonarQube) generate thousands of minor styling warnings that overwhelm development teams and fail to prioritize issues based on business impact.

Main Objectives

The Sentinel platform is designed to establish a predictive engineering intelligence layer directly above the existing development stack. The primary objectives are to:

1. Ingest multi-source engineering artifacts (commits, pull requests, issues, testing, and documentation).
2. Build a unified Project Knowledge Graph representing the software development lifecycle.
3. Calculate an overall, multi-dimensional Project Health Score across maintainability, delivery, stability, and collaboration.

4. Predict bug-prone files, technical debt hotspots, merge conflict probabilities, sprint completion risks, and developer knowledge bottlenecks.
5. Provide mathematically grounded, natural-language explanations for all predictions using Explainable AI (XAI) and local large language models (LLMs).
6. Generate prioritized, actionable recommendations to mitigate identified risks.

System Scope

The system is structured to progress through two distinct developmental phases, balancing immediate prototype feasibility on local hardware with long-term enterprise scalability:

1. **The Initial Prototype (Mini Project Scope):** Focuses on demonstrating core predictive capability using a localized, CPU-optimized stack. It establishes local Git and GitHub REST API integrations, mines basic software metrics, computes a unified Project Health Score, predicts bug-prone modules and technical debt hotspots, and provides natural-language justifications via a local, quantized large language model. It is explicitly designed to run entirely offline on standard developer hardware, specifically a quad-core processor with 16 GB of RAM.
2. **The Production Platform (Main Project Extension):** Expands the system into an enterprise-wide developer enablement ecosystem. It integrates additional data sources, including GitLab, Azure DevOps, Jira, and Jenkins CI/CD pipelines. The predictive capabilities are scaled to forecast sprint delay probabilities, code churn risks, pull request stability, and merge conflicts. Furthermore, the production platform introduces multi-project portfolio dashboards, security vulnerability predictions, and an automated weekly report generator.

Target Audience

The platform serves multiple distinct user groups within a modern software organization, ensuring that predictive insights are tailored to specific operational roles:

- **Software Engineers and Individual Contributors:** These users leverage the platform to identify risky, highly coupled code paths and anticipate potential merge conflicts before committing code. The system provides them with low-friction, localized diagnostics that highlight co-change patterns and recommend specific refactoring tasks, helping them maintain code quality without introducing administrative overhead.
- **Engineering Managers and Tech Leads:** These stakeholders use the platform to monitor team-level sprint completion risks, identify knowledge bottlenecks

(high bus factors), and detect developer workload imbalances. The platform provides the objective telemetry required to unblock developers and facilitate balanced code reviews without resorting to punitive, individual-level metrics tracking.

- **Product Owners and Project Managers:** This group relies on predictive sprint analytics to adjust backlog priorities, estimate delivery delays, and align release roadmaps with actual repository evolution rather than subjective story point estimations.
- **Engineering Executives and CTOs:** Executive leadership utilizes high-level portfolio dashboards to monitor long-term technical debt growth, track organizational health trends, identify off-boarding risks, and evaluate the operational impact of code quality investments.
- **Academic Evaluators and Researchers:** In academic contexts, the platform serves as an open-source tool to validate empirical software engineering theories, study collaborative student team dynamics, and test new machine learning models using public repositories.

Modules

The platform's features are partitioned across 14 functional modules. The table below lists the modules, their functional descriptions, and their implementation phases:

Module Identifier	Module Name	Functional Description	Implementation Phase
Module 1	User & Organization Management	Handles authentication, organization creation, team structures, and role-based access control.	Mini Project
Module 2	Repository Integration	Establishes connectors for GitHub, GitLab, and local Git repositories to import commits, branches, and PRs.	Mini Project & Main Project

Module Identifier	Module Name	Functional Description	Implementation Phase
Module 3	Issue Tracker Integration	Ingests Milestones, tasks, sprints, and labels from systems like Jira, Azure DevOps, and Linear.	Main Project
Module 4	Repository Mining Engine	Extracts code complexity, contributor activity, churn, code ownership, and dependency changes.	Mini Project
Module 5	Project Knowledge Graph	Constructs relationships between developers, files, modules, issues, pull requests, and documentation.	Mini Project & Main Project
Module 6	Project Health Engine	Generates a unified Project Health Score across maintainability, delivery, stability, and collaboration.	Mini Project
Module 7	Bug Prediction Engine	Predicts which modules or files are most likely to contain future defects.	Mini Project
Module 8	Technical Debt Prediction	Quantifies technical debt hotspots and ranks modules needing refactoring.	Mini Project

Module Identifier	Module Name	Functional Description	Implementation Phase
Module 9	Sprint Risk Prediction	Forecasts delayed or incomplete sprints based on velocity trends and task-code mismatches.	Main Project
Module 10	Merge Conflict Prediction	Evaluates parallel branch modifications to identify high probability merge conflicts.	Main Project
Module 11	Developer Knowledge Risk	Identifies critical single-developer dependencies and high bus factors.	Main Project
Module 12	Recommendation Engine	Synthesizes prioritized recommendations for refactoring, testing, and workload re-allocation.	Mini Project & Main Project
Module 13	Explainable AI (XAI)	Generates mathematically grounded and natural-language justifications for predictions.	Mini Project
Module 14	Engineering Dashboard	Visualizes overall health trends, risk heatmaps, module rankings, and contributor analytics.	Mini Project

User Roles

To maintain strict data security and prevent the platform from becoming a tool for punitive employee monitoring, Sentinel enforces granular, role-based access levels. These access controls align permission boundaries with organizational roles:

User Role	Ingestion & System Configuration	Project-Level Telemetry Access	Developer-Specific Analytics	Dashboard and Export Access
System Administrator	Full permission; configures repository integrations and system settings.	Read and write access to all project metrics and configuration logs.	Permitted to view configuration metadata; no access to behavioral social charts.	Complete access.
Engineering Manager	Permitted to connect assigned team repositories.	Read-only access to active team dashboards, sprint risks, and knowledge maps.	Access restricted to team-level collaboration patterns and bus factor risks.	Can view team-level dashboards and generate sprint reports.
Software Engineer	Strictly denied.	Read-only access to assigned repositories and branch diagnostics.	Permitted to view personal code quality, co-change risks, and assigned tasks.	Access restricted to localized developer-centric views.

System Ownership

The system is configured as an academic and developer research platform owned by the individual investigator, designed for open-source distribution and offline verification on personal developer hardware.

Industry/Domain

Software engineering diagnostics, DevOps telemetry, and IT project risk management.

Data Collection Contacts

- **Abidh Habeeb**, Founder & CEO of Incial
 - *Contact*: +91 80751 01530
 - *Contribution*: Provided valuable insights for this project through expert interviews and by answering critical domain-specific questions regarding behavioral code analysis.



Questionnaire for Data Collection

1. **How does your team currently track the health and potential risks of software projects?**

Answer: We use predictive AI models to forecast risks before they occur.

2. **How is technical debt prioritized against building new features in your workflow?**

Answer: We allocate a fixed percentage of every sprint to tech debt.

3. **How do you currently handle or anticipate merge conflicts?**

Answer: We communicate heavily in daily standups to prevent overlap, alongside utilizing predictive analytics for parallel branch modifications.

4. **How does your organization identify developer knowledge bottlenecks (e.g., a high "bus factor")?**

Answer: We actively track individual code ownership and dependency metrics within our repository mining framework.

5. **How does your team handle the warnings generated by static analysis tools (like SonarQube)?**

Answer: We review and fix every warning before merging, though traditional static analysis frequently generates excessive styling noise that lacks true business prioritization.

6. **When a sprint is delayed or fails, what is usually the primary cause?**

Answer: Sprint failures typically stem from a compound mixture of unexpected technical debt accumulation, developer workload imbalances, and task-code mismatches.

7. **How do developers on your team generally perceive the use of Git-based metrics (like PR volumes or lines of code) for evaluation?**

Answer: Developers express deep skepticism and anxiety toward individual metrics, noting that metrics tracking often triggers gamification (such as empty commits or trivial PR splits) and creates micromanagement pressure. (*Self-Assessed / Inferred*)

- 8. For your team to trust an AI prediction (e.g., "this module will cause a bug"), what is the most critical requirement?**

Answer: High accuracy scores on a benchmark dataset.

- 9. Which predictive metric would be most valuable on a daily engineering dashboard?**

Answer: Both bug-prone module alerts/ranked technical debt hotspots and sprint completion risks/merge conflict probabilities are vital for holistic project health

- 10. What is your primary concern regarding an AI tool analyzing your organization's code and collaboration habits?**

Answer: Both external data leakage concerns (preferring local execution) and the risk of metrics being weaponized for punitive, individual-level micromanagement are primary organizational concerns.